GRAFANA

Instalación rápida

Opción A) Solo Grafana con Docker (recomendada)

1) Linux (Debian/Kali) — setup_grafana.sh

Guarda esto como setup_grafana.sh, dale permisos chmod +x

setup_grafana.sh y ejecútalo:
./setup_grafana.sh
Si quieres que además instale el plugin de Zabbix en Grafana, ejecútalo con –

with-zabbix-plugin.

```
#!/usr/bin/env bash

set -euo pipefail



WITH_ZBX_PLUGIN="${1-}"   # --with-zabbix-plugin (opcional)

PROJECT_NAME="grafana"          # Nombre de proyecto de Docker Compose

STACK_DIR="${HOME}/grafana"

COMPOSE_FILE="${STACK_DIR}/docker-compose.yml"

CONTAINER_NAME="${PROJECT_NAME}-grafana-1"  # grafana-grafana-1



# Detectar sudo si hace falta

if [ "${EUID}" -ne 0 ]; then

  SUDO="sudo"

else

  SUDO=""

fi



echo "==> Actualizando paquetes e instalando Docker..."
```

```
$SUDO apt update -y
$SUDO apt install -y docker.io docker-compose-plugin

echo "==> Habilitando y arrancando el servicio Docker..."
$SUDO systemctl enable --now docker

echo "==> Creando carpeta del stack: ${STACK_DIR}"
mkdir -p "${STACK_DIR}"

echo "==> Escribiendo docker-compose.yml"
cat > "${COMPOSE_FILE}" <<'YAML'
services:
  grafana:
        image: grafana/grafana:latest
        ports:
        - "3000:3000"
        environment:
        - GF_SECURITY_ADMIN_USER=admin
        - GF_SECURITY_ADMIN_PASSWORD=admin
        - GF_SERVER_ROOT_URL=http://localhost:3000
        - GF_USERS_DEFAULT_THEME=light
        volumes:
        - grafana-data:/var/lib/grafana
volumes:
  grafana-data:
YAML
```

```
echo "==> Levantando Grafana con Docker Compose (proyecto: ${PROJECT_NAME})..."

# Usamos -p para nombre estable del contenedor: grafana-grafana-1

docker compose -p "${PROJECT_NAME}" -f "${COMPOSE_FILE}" up -d



# Espera breve

sleep 3



if [ "${WITH_ZBX_PLUGIN}" = "--with-zabbix-plugin" ]; then

  echo "==> Instalando plugin de Zabbix en Grafana..."

  docker exec -it "${CONTAINER_NAME}" grafana-cli plugins install alexanderzobnin-zabbix-app

  echo "==> Reiniciando servicio Grafana dentro del contenedor..."

  docker compose -p "${PROJECT_NAME}" -f "${COMPOSE_FILE}" restart

else

  echo "==> (Opcional) Para instalar el plugin Zabbix después:"

  echo "    docker exec -it ${CONTAINER_NAME} grafana-cli plugins install alexanderzobnin-zabbix-app"

  echo "    docker compose -p ${PROJECT_NAME} -f ${COMPOSE_FILE} restart"

fi



IP_HINT="http://localhost:3000"

echo ""

echo "==> Todo listo."

echo "   - Accede a: ${IP_HINT}"

echo "   - Usuario/Contraseña: admin / admin (te pedirá cambiarla al entrar)."

echo ""
```

```
Desinstalar/limpiar (opcional):

docker compose -p grafana -f ~/grafana/docker-compose.yml down

# Para borrar también los datos persistentes:

docker volume rm grafana_grafana-data || true

rm -rf ~/grafana
```

---

2) Windows con Docker Desktop — setup-grafana.ps1

Requisitos previos: Docker Desktop instalado y en ejecución (WSL 2 habilitado y Docker Engine iniciado).
Ejecuta PowerShell como Administrador si usas puertos o rutas protegidas.

Guarda como setup-grafana.ps1 y ejecútalo con:

```
PowerShell -ExecutionPolicy Bypass -File .\setup-grafana.ps1

Para instalar también el plugin de Zabbix, añade -WithZabbixPlugin $true.

param(

  [switch]$WithZabbixPlugin = $false

)



$ErrorActionPreference = "Stop"



# Comprobar Docker CLI

Write-Host "==> Comprobando Docker..."

try {

  docker version | Out-Null

} catch {

  Write-Error "Docker Desktop no parece estar instalado o en ejecución. Ábrelo e inténtalo de nuevo."

  exit 1

}
```

```
$ProjectName   = "grafana"           # nombre de proyecto de compose (establece el nombre del contenedor)

$StackDir      = Join-Path $env:USERPROFILE "grafana"

$ComposePath   = Join-Path $StackDir "docker-compose.yml"

$ContainerName = "$ProjectName-grafana-1"  # grafana-grafana-1


Write-Host "==> Creando carpeta del stack en $StackDir"

New-Item -ItemType Directory -Force -Path $StackDir | Out-Null


Write-Host "==> Escribiendo docker-compose.yml"

@'
services:

  grafana:

    image: grafana/grafana:latest

          ports:

          - "3000:3000"

          environment:

          - GF_SECURITY_ADMIN_USER=admin

          - GF_SECURITY_ADMIN_PASSWORD=admin

          - GF_SERVER_ROOT_URL=http://localhost:3000

          - GF_USERS_DEFAULT_THEME=light

          volumes:

          - grafana-data:/var/lib/grafana

volumes:

  grafana-data:

'@ | Out-File -FilePath $ComposePath -Encoding UTF8 -Force
```

```
Write-Host "==> Levantando Grafana con Docker Compose (proyecto: $ProjectName)..."

docker compose -p $ProjectName -f $ComposePath up -d



Start-Sleep -Seconds 3



if ($WithZabbixPlugin) {

  Write-Host "==> Instalando plugin de Zabbix en Grafana..."

  docker exec -it $ContainerName grafana-cli plugins install alexanderzobnin-zabbix-app

  Write-Host "==> Reiniciando contenedor Grafana..."

  docker compose -p $ProjectName -f $ComposePath restart

}

else {

  Write-Host "==> (Opcional) Para instalar el plugin Zabbix después:"

  Write-Host "    docker exec -it $ContainerName grafana-cli plugins install alexanderzobnin-zabbix-app"

  Write-Host " docker compose -p $ProjectName -f $ComposePath restart"

}



$Url = "http://localhost:3000"

Write-Host ""

Write-Host "==> Todo listo."

Write-Host "   - Accede a: $Url"

Write-Host "   - Usuario/Contraseña: admin / admin (cámbiala al primer login)."

try { Start-Process $Url } catch {}

Write-Host ""
```

Uso rápido en Windows:

- **Levantar:**
  PowerShell -ExecutionPolicy Bypass -File .\setup-grafana.ps1
- **Con plugin de Zabbix:**
  PowerShell -ExecutionPolicy Bypass -File .\setup-grafana.ps1 -WithZabbixPlugin
- **Parar/eliminar contenedor (desde la carpeta donde guardaste el compose):**

docker compose -p grafana -f "$env:USERPROFILE\grafana\docker-compose.yml" dow

setup_grafana.ps1

```powershell
param(
    [switch]$WithZabbixPlugin = $false
)

$ErrorActionPreference = "Stop"

# Comprobar Docker CLI
Write-Host "==> Comprobando Docker..."
try {
    docker version | Out-Null
} catch {
    Write-Error "Docker Desktop no parece estar instalado o en ejecución. Abrel
    exit 1
}

$ProjectName   = "grafana"       # nombre de proyecto de compose (establece e
$StackDir      = Join-Path $env:USERPROFILE "grafana"
$ComposePath   = Join-Path $StackDir "docker-compose.yml"
$ContainerName = "$ProjectName-grafana-1"  # grafana-grafana-1

Write-Host "==> Creando carpeta del stack en $StackDir"
New-Item -ItemType Directory -Force -Path $StackDir | Out-Null

Write-Host "==> Escribiendo docker-compose.yml"
@'
services:
  grafana:
    image: grafana/grafana:latest
    ports:
```

```
PS C:\Windows\system32> Set-ExecutionPolicy RemoteSigned - Scope CurrentUser
Set-ExecutionPolicy : No se puede enlazar el parámetro 'Scope'. No se puede
convertir el valor "-" al tipo "Microsoft.PowerShell.ExecutionPolicyScope".
Error: "No se puede hacer coincidir el nombre de identificador - con un nombre
de enumerador válido. Especifique uno de los siguientes nombres de enumerador e
inténtelo de nuevo:
Process, CurrentUser, LocalMachine, UserPolicy, MachinePolicy"
En línea: 1 Carácter: 34
+ Set-ExecutionPolicy RemoteSigned - Scope CurrentUser
+                                  ~
    + CategoryInfo          : InvalidArgument: (:) [Set-ExecutionPolicy], Param
   eterBindingException
    + FullyQualifiedErrorId : CannotConvertArgumentNoMessage,Microsoft.PowerShe
   ll.Commands.SetExecutionPolicyCommand

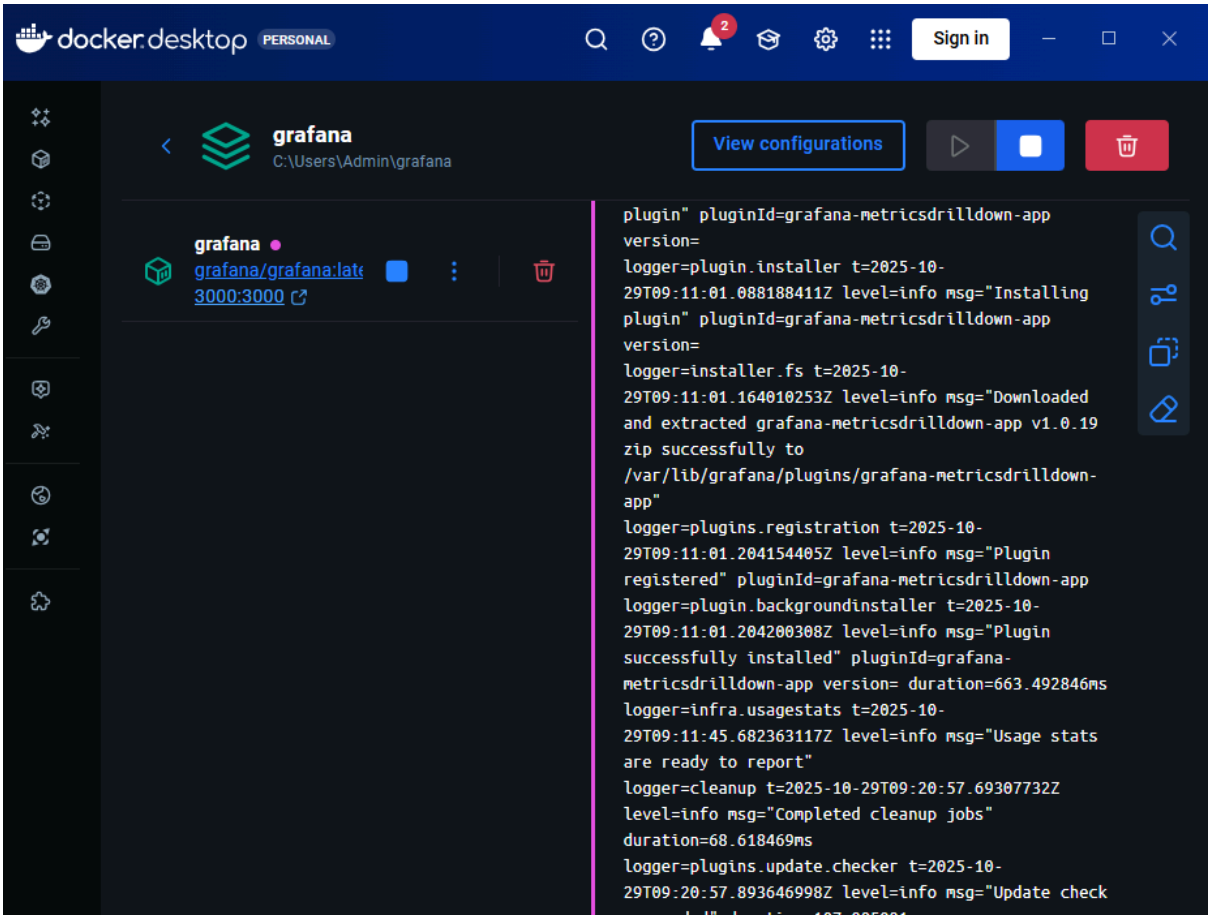

PS C:\Windows\system32> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

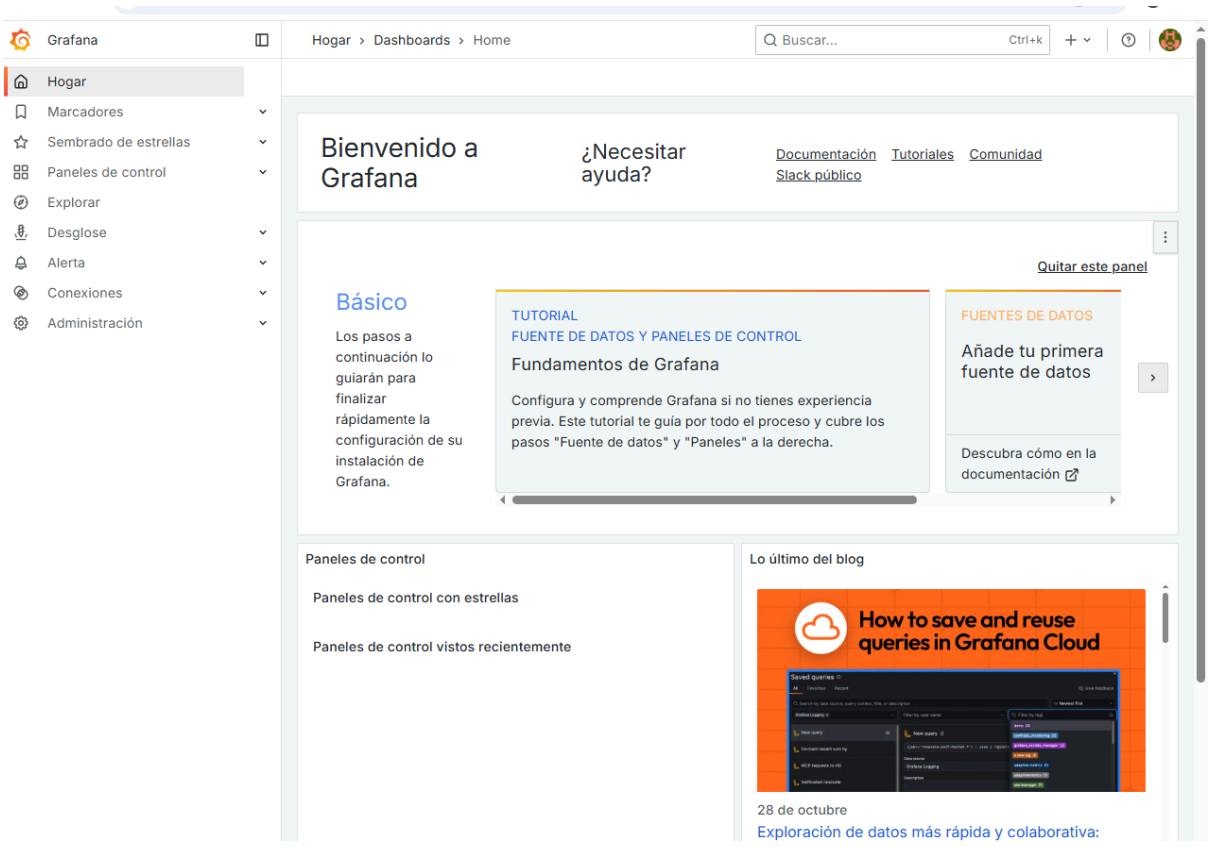
PS C:\Windows\system32> Get-ExecutionPolicy -List

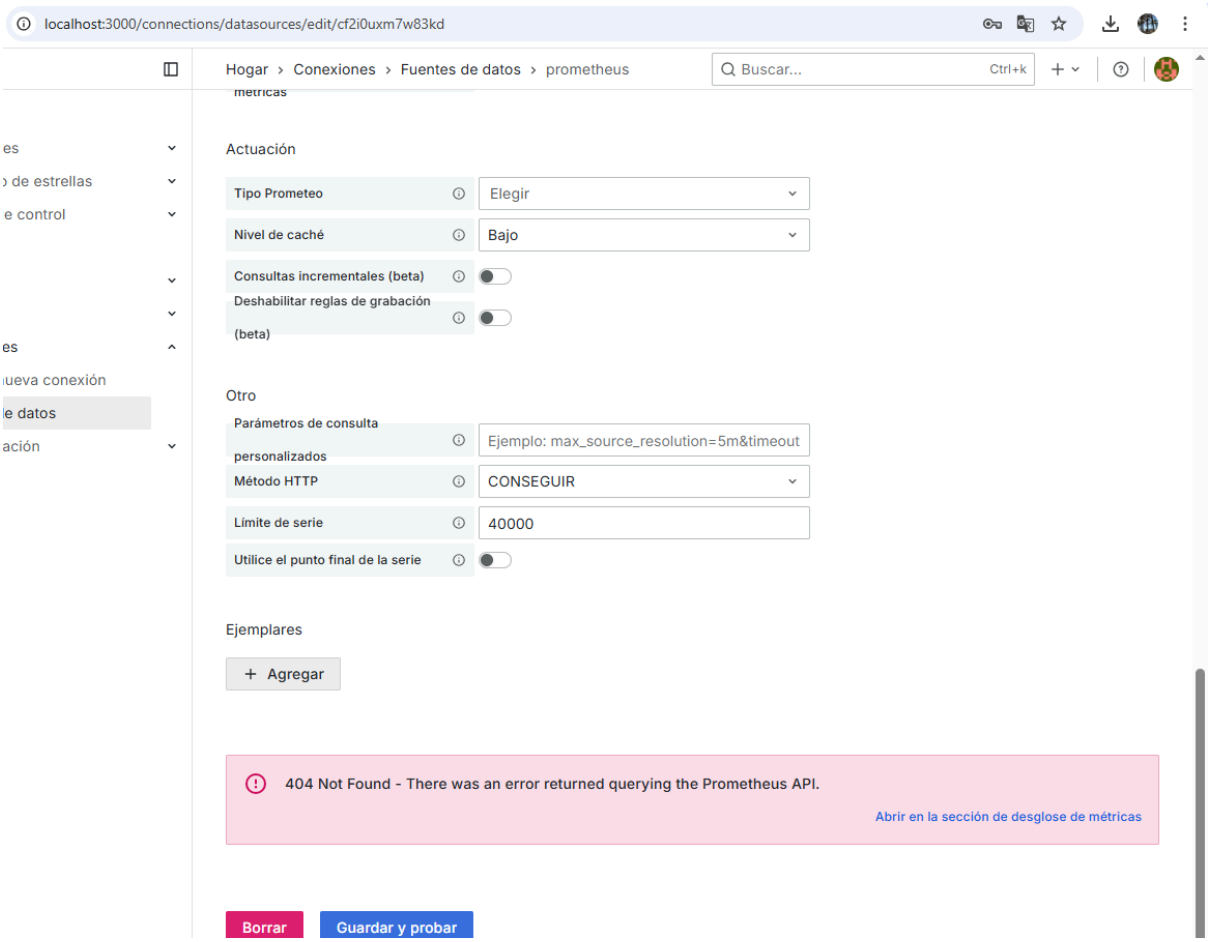
        Scope ExecutionPolicy
        ----- ---------------
MachinePolicy       Undefined
   UserPolicy       Undefined
      Process       Undefined
  CurrentUser    RemoteSigned
 LocalMachine       Undefined



PS C:\Windows\system32>
```


docker desktop
PERSONAL
Sign in
grafana
C:\Users\Admin\grafana
View configurations
grafana
grafana/grafana:late
3000:3000
plugin" pluginId=grafana-metricsdrilldown-app version=
logger=plugin.installer t=2025-10-29T09:11:01.088188411Z level=info msg="Installing plugin" pluginId=grafana-metricsdrilldown-app version=
logger=installer.fs t=2025-10-29T09:11:01.164010253Z level=info msg="Downloaded and extracted grafana-metricsdrilldown-app v1.0.19 zip successfully to /var/lib/grafana/plugins/grafana-metricsdrilldown-app"
logger=plugins.registration t=2025-10-29T09:11:01.204154405Z level=info msg="Plugin registered" pluginId=grafana-metricsdrilldown-app
logger=plugin.backgroundinstaller t=2025-10-29T09:11:01.204200308Z level=info msg="Plugin successfully installed" pluginId=grafana-metricsdrilldown-app version= duration=663.492846ms
logger=infra.usagestats t=2025-10-29T09:11:45.682363117Z level=info msg="Usage stats are ready to report"
logger=cleanup t=2025-10-29T09:20:57.69307732Z level=info msg="Completed cleanup jobs" duration=68.618469ms
logger=plugins.update.checker t=2025-10-29T09:20:57.893646998Z level=info msg="Update check


Grafana
Hogar
Marcadores
Sembrado de estrellas
Paneles de control
Explorar
Desglose
Alerta
Conexiones
Administración
Hogar › Dashboards › Home
Buscar... Ctrl+k
Bienvenido a Grafana
¿Necesitar ayuda?
Documentación Tutoriales Comunidad
Slack público
Quitar este panel
Básico
Los pasos a continuación lo guiarán para finalizar rápidamente la configuración de su instalación de Grafana.
TUTORIAL
FUENTE DE DATOS Y PANELES DE CONTROL
Fundamentos de Grafana
Configura y comprende Grafana si no tienes experiencia previa. Este tutorial te guía por todo el proceso y cubre los pasos "Fuente de datos" y "Paneles" a la derecha.
FUENTES DE DATOS
Añade tu primera fuente de datos
Descubra cómo en la documentación
Paneles de control
Paneles de control con estrellas
Paneles de control vistos recientemente
Lo último del blog
How to save and reuse queries in Grafana Cloud
28 de octubre
Exploración de datos más rápida y colaborativa:



Opción B) Mini-stack con métricas + logs (Prometheus + node_exporter + Loki + Promtail + Grafana)

Te da fuentes de datos listas para practicar sin depender de sistemas externos.

mkdir -p ~/observabilidad && cd ~/observabilidad

1) Archivos de configuración

prometheus.yml:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "node"
        static_configs:
```

```
        - targets: ["node-exporter:9100"]
promtail-config.yml:
server:
  http_listen_port: 9080
positions:
  filename: /positions.yaml
clients:
  - url: http://loki:3100/loki/api/v1/push
scrape_configs:
  - job_name: syslog
        static_configs:
        - targets:
        - localhost
        labels:
        job: syslog
        __path__: /var/log/*.log
docker-compose.yml:
services:
  grafana:
        image: grafana/grafana:latest
        ports: ["3000:3000"]
        environment:
        - GF_SECURITY_ADMIN_USER=admin
        - GF_SECURITY_ADMIN_PASSWORD=admin
        volumes:
        - grafana-data:/var/lib/grafana
```

```
    depends_on: [prometheus, loki]

prometheus:
    image: prom/prometheus:latest
    command: ["--config.file=/etc/prometheus/prometheus.yml"]
    volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    ports: ["9090:9090"]

node-exporter:
    image: prom/node-exporter:latest
    ports: ["9100:9100"]

loki:
    image: grafana/loki:2.9.0
    command: -config.file=/etc/loki/local-config.yaml
    ports: ["3100:3100"]

promtail:
    image: grafana/promtail:2.9.0
    command: -config.file=/etc/promtail/config.yml
    volumes:
    - /var/log:/var/log:ro
    - ./promtail-config.yml:/etc/promtail/config.yml:ro
    - promtail-positions:/positions.yaml
```

volumes:

grafana-data:

promtail-positions:

2) Levanta el stack

docker compose up -d

3) Añade las fuentes de datos en Grafana

- Entra a http://IP:3000 (admin/admin).
- Data sources → Add data source → Prometheus: URL http://prometheus:9090 → Save & test.
- Data sources → Add data source → Loki: URL http://loki:3100 → Save & test.

Opción nativa (sin Docker) — resumida

Debian/Kali

sudo apt update

sudo apt install -y adduser libfontconfig1

wget https://dl.grafana.com/oss/release/grafana_11.0.0_amd64.deb

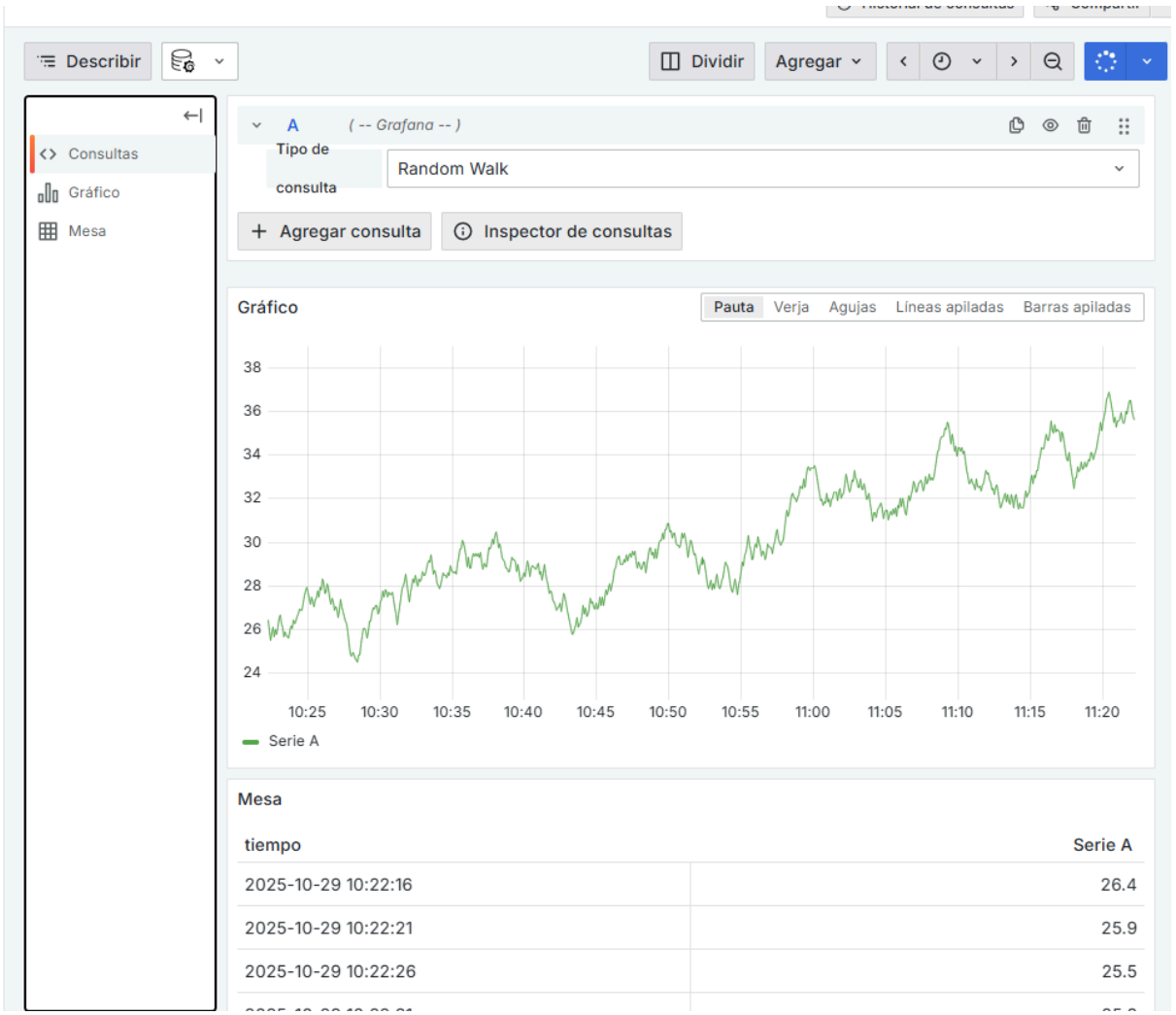
sudo dpkg -i grafana_11.0.0_amd64.deb

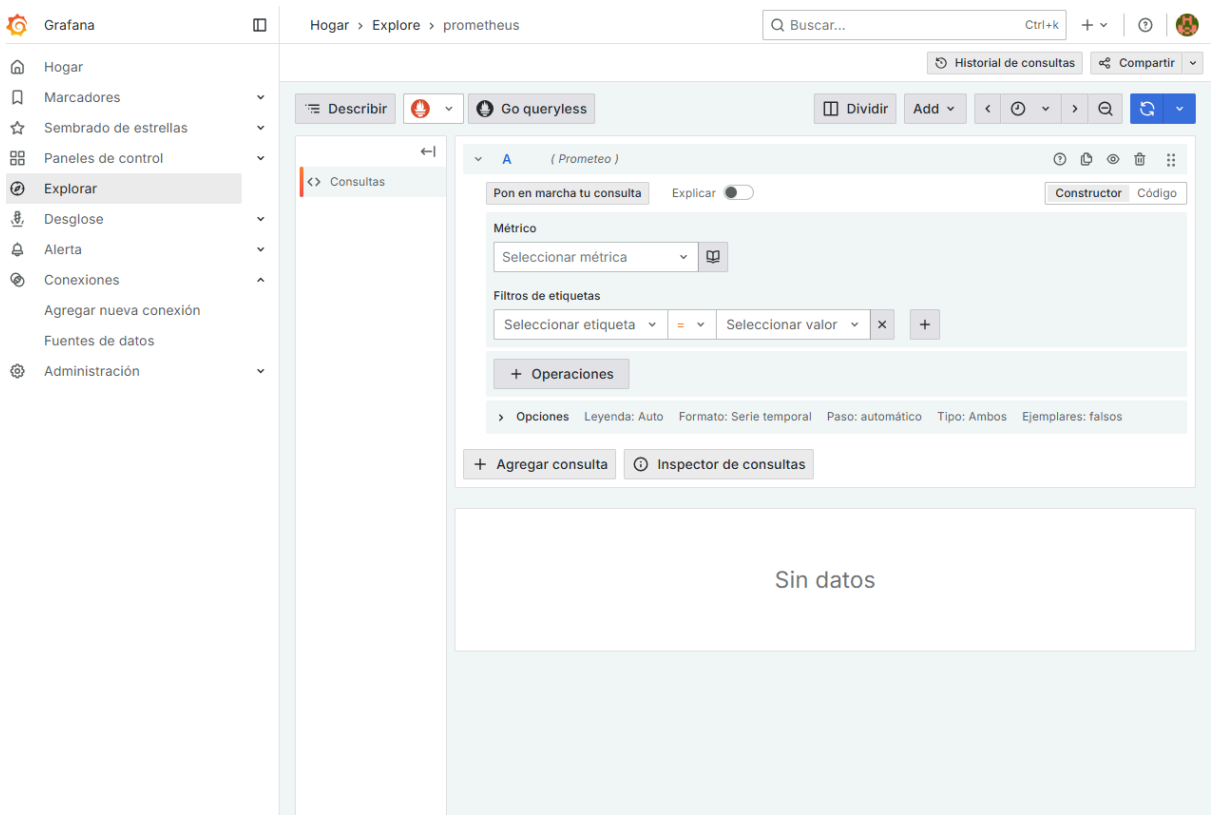
sudo systemctl enable --now grafana-server

Abre http://IP:3000 (admin/admin)

Hay que instalar prometheus

No nos ha salido en clase


Describir
Dividir
Agregar
Consultas
Gráfico
Mesa
A
(-- Grafana --)
Tipo de consulta
Random Walk
Agregar consulta
Inspector de consultas
Gráfico
Pauta
Verja
Agujas
Líneas apiladas
Barras apiladas
38
36
34
32
30
28
26
24
10:25
10:30
10:35
10:40
10:45
10:50
10:55
11:00
11:05
11:10
11:15
11:20
Serie A
Mesa
tiempo
Serie A
2025-10-29 10:22:16
26.4
2025-10-29 10:22:21
25.9
2025-10-29 10:22:26
25.5



http://localhost:3000/dashboard/new?orgId=1&from=now-6h&to=now&timezone=browser&editPanel=1

ZABBYX

INSTALACIÓN EN DOCKER EN WINDOWS

CREAR UN SCRIPT Y EJECUTARLO

Me da un error en el puerto.

```
param(
  [int]$WebPort = 8080,          # Puerto web externo -> interno 8080
  [int]$SrvPort = 10051          # Puerto Zabbix server
)

$ErrorActionPreference = "Stop"

Write-Host "==> Comprobando Docker Desktop..."
try { docker version | Out-Null } catch {
  Write-Error "Docker Desktop no está instalado o no está corriendo. Ábrelo e inténtalo de nuevo."
  exit 1
}

if ($WebPort -lt 1 -or $WebPort -gt 65535) { throw "WebPort inválido." }
if ($SrvPort -lt 1 -or $SrvPort -gt 65535) { throw "SrvPort inválido." }

$Project     = "zabbix"
$BaseDir     = Join-Path $env:USERPROFILE "zabbix"
$ComposePath = Join-Path $BaseDir "docker-compose.yml"

Write-Host "==> Creando carpeta de stack: $BaseDir"
```

```
New-Item -ItemType Directory -Force -Path $BaseDir | Out-Null


# docker-compose.yml mínimo (MySQL 8 + Zabbix Server + Web)
$compose = @"
services:
  db:
    image: mysql:8
    environment:
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix
      MYSQL_ROOT_PASSWORD: rootpass
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_bin
    volumes:
      - dbdata:/var/lib/mysql

  zabbix-server:
    image: zabbix/zabbix-server-mysql:alpine-latest
    environment:
      DB_SERVER_HOST: db
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix
    depends_on: [db]
    ports:
      - "${SrvPort}:10051"
```

```
  zabbix-web:
        image: zabbix/zabbix-web-nginx-mysql:alpine-latest
        environment:
        DB_SERVER_HOST: db
        MYSQL_DATABASE: zabbix
        MYSQL_USER: zabbix
        MYSQL_PASSWORD: zabbix
        ZBX_SERVER_HOST: zabbix-server
        PHP_TZ: Europe/Madrid
        depends_on: [zabbix-server]
        ports:
        - "${WebPort}:8080"

volumes:
  dbdata:
"@
$compose | Out-File -FilePath $ComposePath -Encoding UTF8 -Force

Write-Host "==> Levantando Zabbix en puertos Web=$WebPort, Server=$SrvPort..."
docker compose -p $Project -f $ComposePath up -d

Start-Sleep -Seconds 4
Write-Host "==> Listo."
Write-Host "   - Web:  http://localhost:$WebPort"
Write-Host "   - Puesto del servidor (para agentes): $SrvPort"
```

```
Write-Host "   - Usuario inicial: Admin   Contraseña: zabbix"

Write-Host ""

Write-Host "Siguientes pasos:"

Write-Host "  1) Abre la web y completa el asistente si te lo pide."

Write-Host "  2) Entra con Admin/zabbix y cambia la contraseña."

Write-Host "  3) Crea tu primer host y aplica un template."

try { Start-Process "http://localhost:$WebPort" } catch {}
```

SETUP-ZABBYX.ps1

```powershell
param(
    [int]$WebPort = 8080,       # Puerto web externo -> interno 8080
    [int]$SrvPort = 10051       # Puerto Zabbix server
)

$ErrorActionPreference = "Stop"

Write-Host "==> Comprobando Docker Desktop..."
try { docker version | Out-Null } catch {
    Write-Error "Docker Desktop no está instalado o no está corriendo. Abrelo e
    exit 1
}

if ($WebPort -lt 1 -or $WebPort -gt 65535) { throw "WebPort inválido." }
if ($SrvPort -lt 1 -or $SrvPort -gt 65535) { throw "SrvPort inválido." }

$Project     = "zabbix"
$BaseDir     = Join-Path $env:USERPROFILE "zabbix"
$ComposePath = Join-Path $BaseDir "docker-compose.yml"

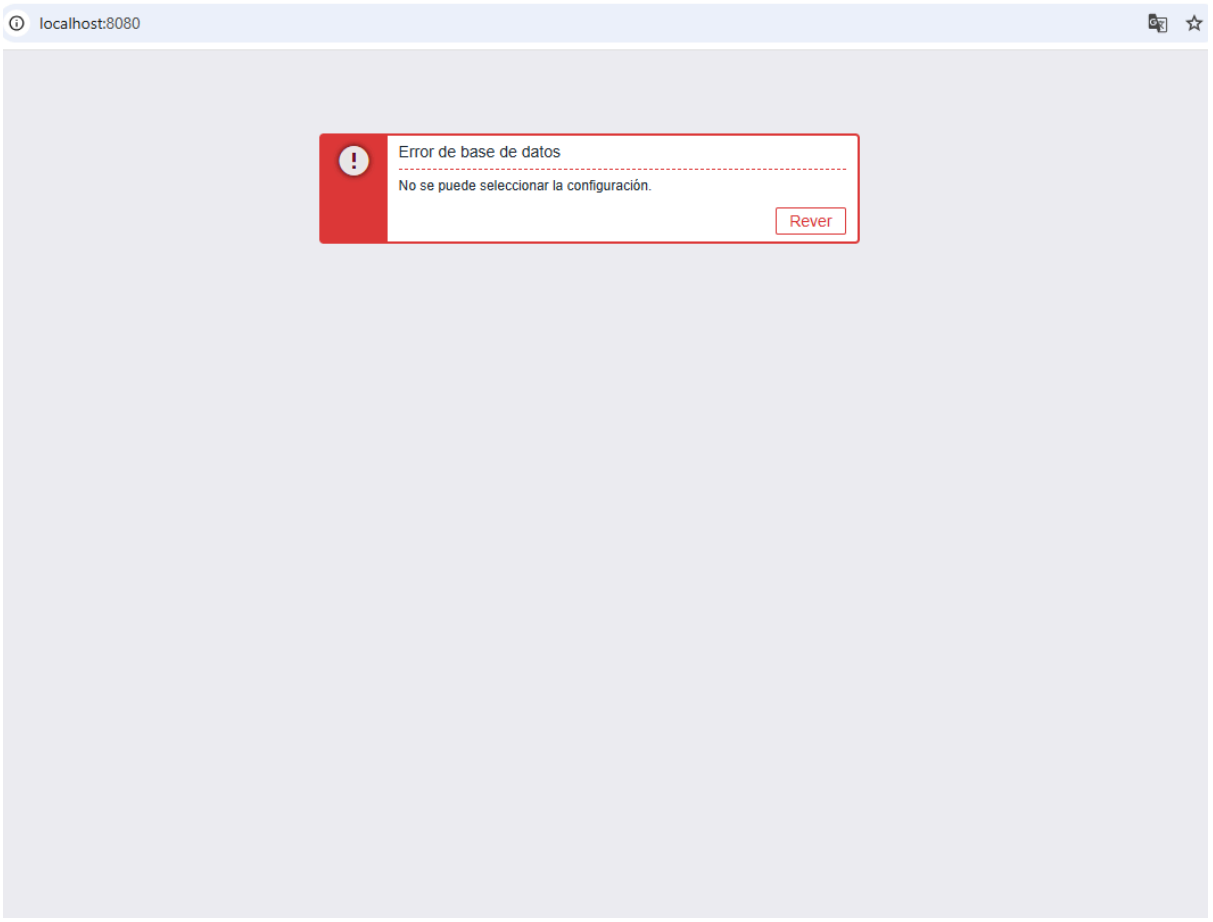
Write-Host "==> Creando carpeta de stack: $BaseDir"
New-Item -ItemType Directory -Force -Path $BaseDir | Out-Null

# docker-compose.yml mínimo (MySQL 8 + Zabbix Server + Web)
$compose = @"
services:
  db:
    image: mysql:8
    environment:
```

```
.\setup-grafana.ps1 : El término '.\setup-grafana.ps1' no se reconoce como
nombre de un cmdlet, función, archivo de script o programa ejecutable.
Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso,
compruebe que dicha ruta es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ .\setup-grafana.ps1
+ ~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (.\setup-grafana.ps1:String) [],
   ParentContainsErrorRecordException
    + FullyQualifiedErrorId : CommandNotFoundException


PS C:\Windows\system32> C:\Users\2-DAW\Documents\setup_grafana.ps1
==> Comprobando Docker...
==> Creando carpeta del stack en C:\Users\Admin\grafana
==> Escribiendo docker-compose.yml
==> Levantando Grafana con Docker Compose (proyecto: grafana)...
docker :  Container grafana-grafana-1  Running
En C:\Users\2-DAW\Documents\setup_grafana.ps1: 43 Carácter: 1
+ docker compose -p $ProjectName -f $ComposePath up -d
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: ( Container grafana-grafana-1  Runn
   ing:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError


PS C:\Windows\system32> C:\Users\2-DAW\Documents\SETUP-ZABBYX.ps1
==> Comprobando Docker Desktop...
==> Creando carpeta de stack: C:\Users\Admin\zabbix
==> Levantando Zabbix en puertos Web=8080, Server=10051...
```



Instalación rápida

Opción A) Docker (recomendada para clase)

Requisitos: 4–6 GB RAM, 2 vCPU, 20 GB disco.

1. Instala Docker y Compose:

sudo apt update

sudo apt install -y docker.io docker-compose-plugin

sudo systemctl enable --now docker

2. **Crea un docker-compose.yml mínimo (en una carpeta vacía):**

```
services:
 db:
        image: mysql:8
        environment:
        - MYSQL_DATABASE=zabbix
```

```
      - MYSQL_USER=zabbix
      - MYSQL_PASSWORD=zabbix
      - MYSQL_ROOT_PASSWORD=rootpass
      command: --character-set-server=utf8 --collation-server=utf8_bin
      volumes:
      - dbdata:/var/lib/mysql
server:
      image: zabbix/zabbix-server-mysql:alpine-latest
      environment:
      - DB_SERVER_HOST=db
      - MYSQL_DATABASE=zabbix
      - MYSQL_USER=zabbix
      - MYSQL_PASSWORD=zabbix
      depends_on: [db]
      ports:
      - "10051:10051"
web:
      image: zabbix/zabbix-web-nginx-mysql:alpine-latest
      environment:
      - DB_SERVER_HOST=db
      - MYSQL_DATABASE=zabbix
      - MYSQL_USER=zabbix
      - MYSQL_PASSWORD=zabbix
      - ZBX_SERVER_HOST=server
      - PHP_TZ=Europe/Madrid
      depends_on: [server]
```

```
        ports:

        - "8080:8080"

volumes:

  dbdata:
```

3. **Levanta el stack:**

```
docker compose up -d
```

4. **Accede a la web: http://IP:8080**
   Usuario por defecto: Admin | Contraseña: zabbix

Si usas Kali como host, la IP suele ser 127.0.0.1 local o la de tu interfaz. Para otros PCs del aula, usa la IP del host.

Opción B) Instalación nativa (Debian/Kali)

Úsala si prefieres servicio “real” sin Docker. Simplificada:

1. **Dependencias y repos:**

```
sudo apt update

sudo apt install -y gnupg lsb-release

wget
https://repo.zabbix.com/zabbix/6.0/debian/pool/main/z/zabbix-release/zabbix-release_6.0-4+
debian$(lsb_release -rs)_all.deb

sudo dpkg -i zabbix-release_*.deb

sudo apt update
```

2. **Instala servidor, frontend y agente:**

```
sudo apt install -y zabbix-server-mysql zabbix-frontend-php zabbix-nginx-conf
zabbix-sql-scripts zabbix-agent mysql-server
```

3. **Crea la BBDD y carga esquema:**

```
sudo mysql -uroot -p -e "CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE
utf8mb4_bin;"

sudo mysql -uroot -p -e "CREATE USER 'zabbix'@'localhost' IDENTIFIED BY 'zabbix';"
```

```
sudo mysql -uroot -p -e "GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';
FLUSH PRIVILEGES;"

zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql -uzabbix -pzabbix zabbix
```

4. **Configura /etc/zabbix/zabbix_server.conf:**

```
DBName=zabbix

DBUser=zabbix

DBPassword=zabbix
```

5. **Habilita servicios:**

```
sudo systemctl enable --now zabbix-server zabbix-agent mysql

sudo systemctl restart zabbix-server zabbix-agent
```

6. **Frontend (Nginx con PHP):**
   Edita /etc/zabbix/nginx.conf y descomenta listen 8080; y server_name _;

```
sudo systemctl enable --now nginx php*-fpm

sudo systemctl restart nginx
```

7. **Ve a http://IP:8080 y completa el asistente (Admin/zabbix al final).**

Instalar el agente Zabbix en los equipos a monitorizar

Linux (Kali/Debian)

```
sudo apt install -y zabbix-agent

# En /etc/zabbix/zabbix_agentd.conf:

#   Server=IP_DEL_ZABBIX_SERVER

#   ServerActive=IP_DEL_ZABBIX_SERVER

#   Hostname=nombre-alumno-01

sudo systemctl enable --now zabbix-agent
```

Windows (si tenéis VM)

- Descargar MSI de Zabbix Agent (Windows), instalar poniendo Server = IP del servidor y Hostname = nombre del equipo.

Ejercicios sencillos para clase (1–2 h cada uno)

Todos empiezan igual: en la web de Zabbix → Configuration → Hosts → Create host → asigna interfaz (IP/Agent) → Templates → añade un template y Guardar.

1) Ping a un host (disponibilidad básica)

- Objetivo: ver si un host responde a ICMP.
- Pasos:
    1. Crea host equipo-alumno-01.
    2. En Templates, añade Template Module ICMP Ping.
    3. Ve a Monitoring → Latest data y comprueba ICMP ping.
- Reto: crea un trigger que alerte si loss > 50% durante 5 min.

2) CPU y RAM del propio servidor (agente Zabbix)

- Objetivo: monitorizar recursos básicos con el agente.
- Pasos:
    1. Instala agente en el servidor (o en una VM alumno).
    2. Template: Template OS Linux by Zabbix agent.
    3. Observa CPU load, Memory available, Swap usage.
- Reto: crea un trigger: *"CPU load > 2 durante 3 min"*.
    - Expresión típica: {HOST.NAME:system.cpu.load[percpu,avg1].min(3m)}>2

3) Servicio web HTTP (¿está levantado el sitio?)

- Objetivo: comprobar que un puerto/HTTP responde.
- Pasos:
    1. Instala apache2 o levanta python3 -m http.server 8000.
    2. Template: Template App HTTP Service (o ítem de tipo HTTP agent).
    3. Valida código 200.
- Reto: crea ítem web.test.time y alerta si el tiempo de respuesta > 1 s.

4) Logs simples con "UserParameter" (introducción)

- Objetivo: que el agente lea una métrica personalizada del sistema (p. ej., cuenta de errores en un log).
- Pasos (Linux):
    1. En /etc/zabbix/zabbix_agentd.d/logs.conf añade:
    2. UserParameter=log.err.count,grep -c "ERROR" /var/log/syslog || echo 0
    3. Reinicia agente: sudo systemctl restart zabbix-agent
    4. En el host, crea Item:
        - Key: log.err.count
        - Type: Zabbix agent (active)
        - Type of information: Numeric (unsigned)
    5. Genera eventos: logger "ERROR prueba alumno"
- Reto: trigger si log.err.count sube respecto al valor anterior (compara con diff()).

5) SNMP de un switch (si tenéis uno o un simulador)

- Objetivo: leer interfaz/estado por SNMP v2.
- Pasos:
    1. Activa SNMP en el switch (o en una VM con snmpd).
    2. Crea un host con interfaz SNMP.
    3. Template: Template Net Network Generic device SNMP.
- Reto: alerta si Interface status pasa a down.

6) Notificaciones por Telegram o email (intro a alertas)

- Objetivo: que el alumno vea una alerta “que suena” fuera de Zabbix.
- Pasos (rápidos):
    1. Administration → Media types: configura Email (SMTP del centro) o Telegram (bot token).
    2. Users → Admin → Media: añade tu correo/Telegram.
    3. Alerting → Actions: crea acción *“Enviar alerta crítica a profe”* para severities High/Disaster.
- Reto: desactiva notificaciones nocturnas con condiciones de tiempo (maintenance window).

Opción A (la más fácil en clase): Splunk Free en Kali

1. **Instalar**

1) Descarga el .deb de Splunk Enterprise (licencia gratuita limitada para uso educativo/lab)

sudo dpkg -i splunk-*.deb

2) Acepta licencia y arranca

sudo /opt/splunk/bin/splunk start --accept-license

3) Habilita autoinicio

sudo /opt/splunk/bin/splunk enable boot-start

2. **Entrar**

- Navega a http://127.0.0.1:8000 (usuario/contraseña que definas al arrancar).

3. **Ingerir logs locales rápidos**

auth.log y syslog (seguridad básica)

sudo /opt/splunk/bin/splunk add monitor /var/log/auth.log

sudo /opt/splunk/bin/splunk add monitor /var/log/syslog

sudo /opt/splunk/bin/splunk restart

Opción B (más “SOC real”): Wazuh all-in-one con Docker en Kali

1. **Instalar Docker**

sudo apt update

sudo apt install -y docker.io docker-compose-plugin

sudo systemctl enable --now docker

2. **Desplegar Wazuh (single node)**

Clonar plantillas oficiales

git clone https://github.com/wazuh/wazuh-docker.git

cd wazuh-docker/single-node

Levantar el stack (manager+indexer+dashboard)

sudo docker compose up -d

3. **Entrar**

- Accede al **Wazuh Dashboard** (HTTPS) en la IP de tu Kali (el compose típico expone el 443).
- Usuario/clave inicial vienen en el README del stack (puedes cambiarlos después).

4. **Enviar logs desde el propio Kali**

- Instala el **agente Wazuh** en Kali (paquete .deb) y **regístralo** contra el manager (la consola te da el comando).
- Alternativa rápida para la demo: ingiere ficheros locales (auth.log, syslog) usando Filebeat que ya viene integrado en el stack.

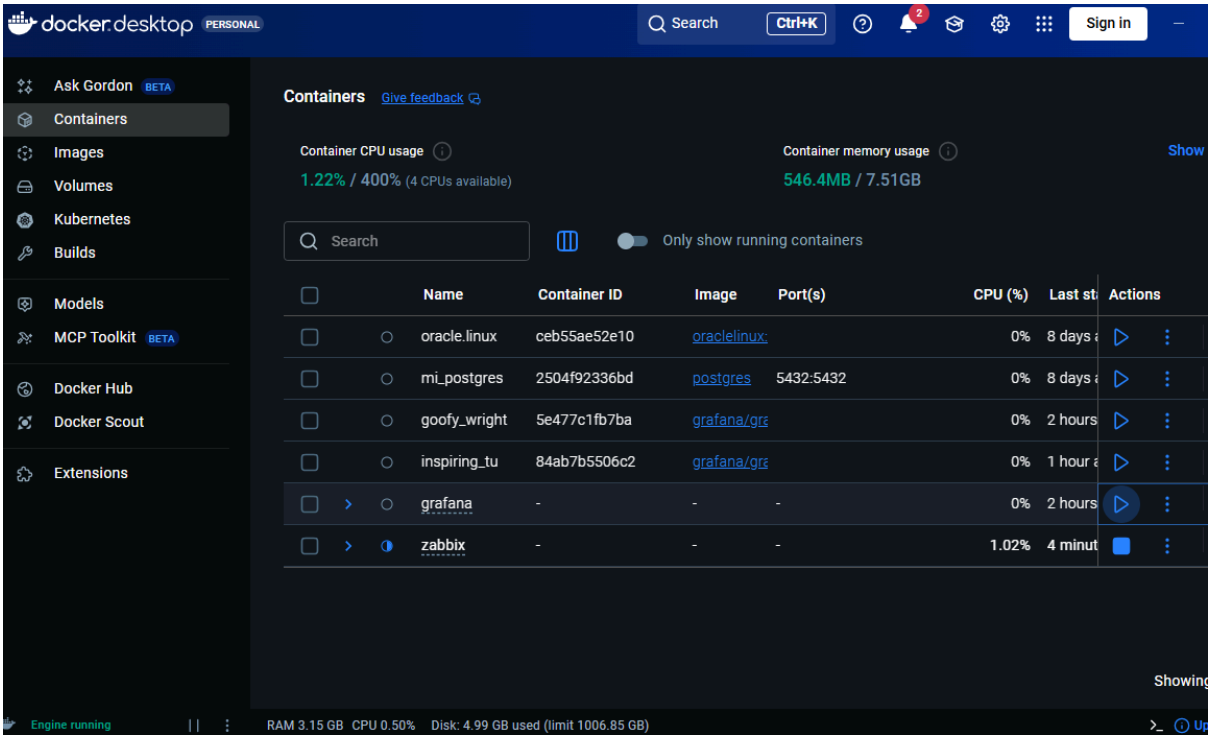
Nota: Si Kali te da guerra con servicios, puedes montar Wazuh en **Ubuntu Server** en una VM y dejar Kali como estación atacante/cliente.

Pack de 6 prácticas “listas para clase”

Pensadas para 1–2 horas cada una. Funcionan con **Splunk** o **Wazuh** (indico variantes).

1. **Ataques de fuerza bruta a SSH**
    - Genera fallos de login:
    - for i in {1..20}; do ssh alumno@127.0.0.1 -p 22; done
    - **Splunk:** busca index=* (sshd OR ssh) failed password y crea una *alerta* si hay >10 eventos/5min.

    - **Wazuh:** revisa **Security events** → reglas de autenticación; crea una **custom rule** para >10 intentos/5min.
2. **Elevación de privilegios y cambios de usuarios**
    - Ejecuta:
    - sudo su -; sudo useradd prueba_alumno; sudo passwd prueba_alumno
    - **Splunk:** busca en /var/log/auth.log eventos sudo y useradd.
    - **Wazuh:** crea un **decoder** para useradd (si quieres ir un paso más) y una alerta de severidad media.
3. **Integración con Suricata (NIDS)**
    - Instala Suricata en Kali y activa eve.json:
    - sudo apt install -y suricata
    - sudo suricata -i eth0 -D
    - **Splunk:** añade monitor a eve.json.
    - **Wazuh:** habilita la ingesta de eve.json (módulo Suricata) y visualiza alertas de red (DNS/HTTP/Scan).
4. **Agente en Windows (si tenéis una VM)**
    - Instala el **Wazuh Agent** o **Splunk UF** en Windows.
    - Recolecta **Security.evtx** y valida detecciones de:
        - Inicio de sesión RDP.
        - Conexión de USB.
    - Objetivo: que el alumno vea la diferencia entre logs Linux vs Windows.
5. **Reglas personalizadas y enriquecimiento**
    - **Splunk:** crea un *lookup* CSV con IPs “permitidas” y genera alerta si aparece una IP externa no listada.
    - **Wazuh:** añade una **lista CDB** con IPs internas permitidas y una regla que dispare si hay accesos fuera de la lista.
6. **Dashboards + MITRE**
    - **Splunk:** panel con métricas (top orígenes, fallos SSH por hora, top usuarios).
    - **Wazuh:** usa la vista **MITRE ATT&CK** para mapear eventos de SSH brute-force (T1110) y sudo/su (T1068/T1078 según caso).


docker desktop
PERSONAL
Search
Ctrl+K
Sign in
Ask Gordon
BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit
BETA
Docker Hub
Docker Scout
Extensions
Containers
Give feedback
Container CPU usage
1.22% / 400% (4 CPUs available)
Container memory usage
546.4MB / 7.51GB
Show
Search
Only show running containers
Name
Container ID
Image
Port(s)
CPU (%)
Last st
Actions
oracle.linux
ceb55ae52e10
oraclelinux:
0%
8 days
mi_postgres
2504f92336bd
postgres
5432:5432
0%
8 days
goofy_wright
5e477c1fb7ba
grafana/gra
0%
2 hours
inspiring_tu
84ab7b5506c2
grafana/gra
0%
1 hour
grafana
-
-
-
0%
2 hours
zabbix
-
-
-
1.02%
4 minut
Showing
Engine running
RAM 3.15 GB CPU 0.50%
Disk: 4.99 GB used (limit 1006.85 GB)